

Retrieval Benchmark: Chunking, Embeddings, and Vector Databases

By Insharah Irfan Nazir

1. Executive Summary

This report evaluates the foundational components of a retrieval system: chunking strategy, embedding model, and vector database. Five embedding models and three vector databases are benchmarked across three chunking strategies—fixed-size, recursive, and semantic—measuring retrieval quality (Recall@k, MRR, Precision) and operational metrics (latency, build time, memory).

The results identify chunking strategy as the highest-leverage decision for retrieval quality, as it determines whether the correct information can be retrieved at all. Embedding model is the second-most critical factor, with multilingual models dramatically outperforming English-only alternatives on non-English text. Vector database choice, when using exact indexes, has no impact on retrieval quality at this scale.

Final System Recommendation: Semantic Chunking + BGE-M3 embedding model + FAISS vector database.

This combination provides the highest retrieval ceiling (0.867 recall@10), the best ranking quality (0.661 MRR), and a zero-dependency, exact-search baseline that ensures retrieval quality is not compromised by database approximations or tie-breaking artifacts. For teams prioritizing operational simplicity and faster query latency without sacrificing recall, Chroma is a viable alternative to FAISS.

2. Methodology

2.1 Corpus

The benchmark corpus consists of 63 documents split into chunks using three strategies. Documents are distributed across three languages:

- English: 43 documents
- Korean: 11 documents
- Urdu: 9 documents

Before embedding, each document's alphabetic-character ratio was computed to flag extraction noise (e.g., OCR artifacts, HTML boilerplate). Ten documents scored notably low (0.63–0.80). Rather than excluding them, these noisy documents were retained to reflect real-world conditions. Reported recall figures therefore include some inherently harder, lower-quality documents; performance on clean text may be modestly higher.

2.2 Evaluation Queries

Forty-five evaluation queries were constructed: 22 in English, 12 in Korean, and 11 in Urdu. Each query has exactly one ground-truth relevant chunk—a held-out sentence drawn from its source document.

Important caveats:

- Korean queries are near-exact substrings of the source text, making retrieval comparatively easier. Model comparisons should be made within Korean rather than against English absolute scores.
- Urdu queries draw from only 3 source documents; strong Urdu scores should be interpreted with this concentration in mind.

2.3 Chunking Strategies Tested



Chunking is the first and most consequential decision in any retrieval pipeline. Three strategies were evaluated:

Strategy	Total Chunks	Description
Fixed-size	6,091	Splits text at a fixed token count regardless of boundaries. Language-blind.
Recursive	7,325	Splits at token count with fallback to sentence boundaries when possible.

Semantic	10,456	Uses embedding similarity to identify natural breakpoints, preserving sentences and paragraphs.
----------	--------	---

2.4 Embedding Models Tested

Five embedding models were selected to represent different design philosophies:

Model	Dimensions	Context	Description
BGE-large-en	1024	512	English-only baseline
BGE-M3	1024	8192	Multilingual, long context
E5-large-v2	1024	512	English-only baseline
multilingual-e5-large	1024	512	Multilingual
Instructor-XL	768	512	Instruction-based

Embedding Speed (GPU, semantic chunking — 10,456 chunks):

Model	Total Time (s)	ms/chunk
multilingual-e5-large	880	84
BGE-M3	975	93
BGE-large-en	1,055	101
E5-large-v2	1,075	103
Instructor-XL	3,807	364

Note: Initial CPU benchmarking showed ~2,173 ms/chunk; all timings reported reflect GPU throughput.

2.5 Vector Databases Tested

Three vector databases were tested with exact (FLAT) indexes to ensure fair comparison. At this scale ($\leq 10,456$ vectors), exact search is fast enough that approximate indexes are unnecessary.

Database	Type	Description
FAISS	Library	Exact brute-force baseline, zero dependencies
Chroma	Embedded DB	Lightweight, persistent storage
Milvus	Server (Docker)	Distributed, designed for horizontal scaling

2.6 Metrics

Metric	Definition
Recall@k	Fraction of true top-k nearest neighbors returned
Precision@k	Fraction of returned results that are relevant
MRR	Mean reciprocal rank of the first correct result
Latency	p50, p95, p99 search times
Build time	Time to index all vectors
Memory	RAM footprint of the index

FAISS exhaustive search (IndexFlatIP) establishes the ground truth for all queries. All recall metrics are measured against this exact baseline.

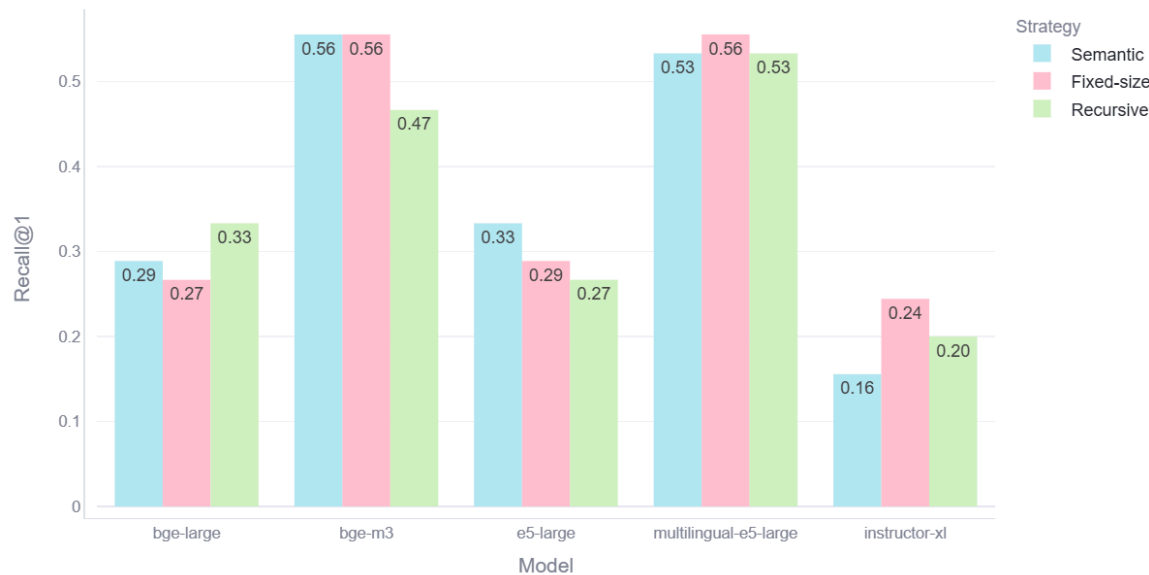
3. Results: Chunking Strategy

Chunking strategy determines the retrieval ceiling. If a correct answer is split across chunk boundaries, no embedding model can retrieve it as a single result. Conversely, overly large chunks may inflate recall@k by ensuring the answer appears somewhere in a retrieved chunk, while simultaneously degrading the downstream LLM's ability to extract the precise answer from irrelevant surrounding text.

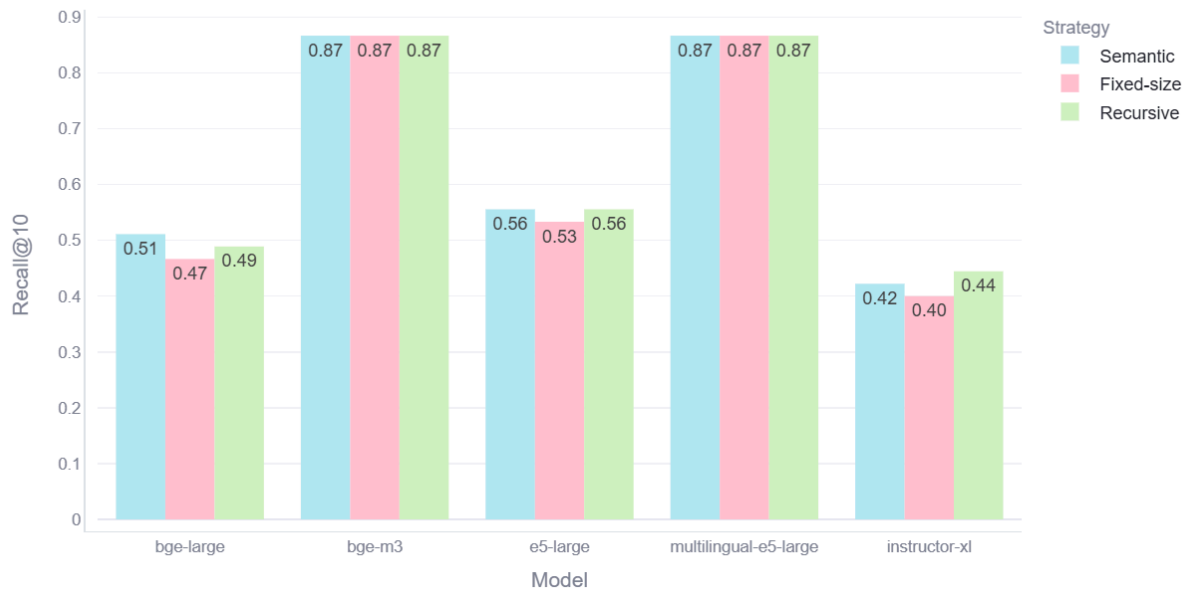
3.1 Chunk Count and Retrieval Quality

Recall@1 by Strategy and Model

Recall@1 — All Models, All Chunking Strategies



Recall@10 — All Models, All Chunking Strategies



Strategy	Chunks	BGE-M3 Recall@1	BGE-M3 Recall@10	BGE-M3 MRR
----------	--------	--------------------	---------------------	---------------

Fixed	6,091	0.556	0.867	0.675
Recursive	7,325	0.467	0.867	0.590
Semantic	10,456	0.556	0.867	0.661

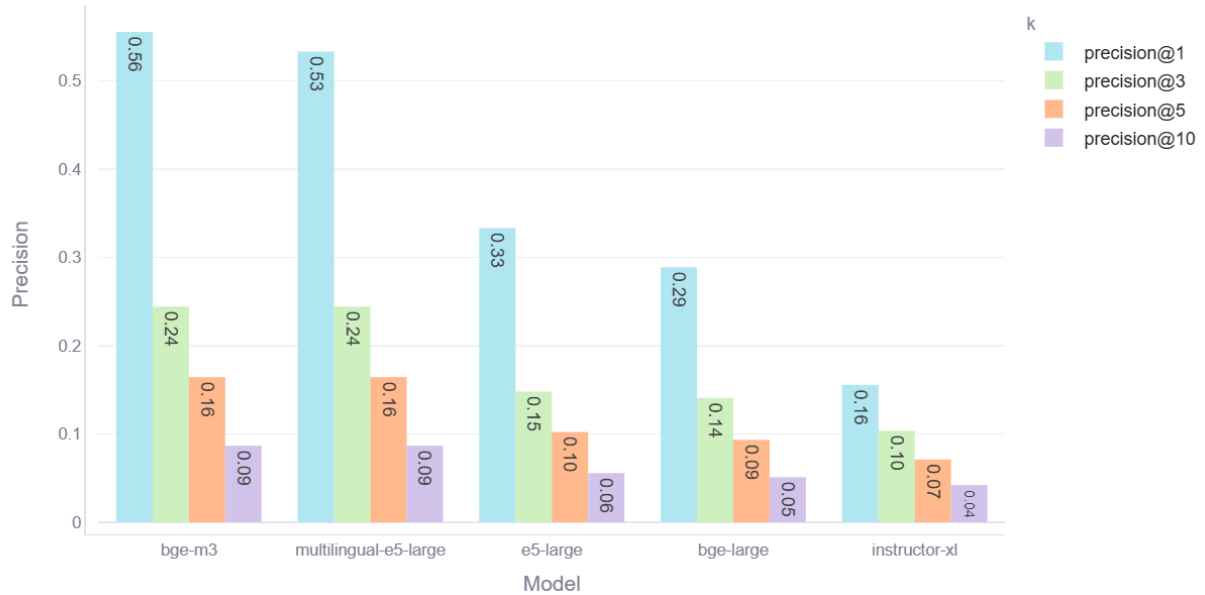
Key observations:

- Fixed-size chunking achieves the best MRR (0.675) and ties for recall@1 (0.556) and recall@10 (0.867). This suggests that even though it is language-blind, it does not systematically fragment answer text more than other strategies for this corpus. However, its lack of semantic boundaries may lead to chunks that mix unrelated topics, reducing precision and making it harder for an LLM to extract the correct answer.
- Semantic chunking produces the most chunks (10,456)—approximately 1.7× more than fixed-size—while matching fixed-size on recall metrics. The higher granularity means each chunk is more focused, which benefits precision: the LLM receives less irrelevant surrounding text, reducing the burden of extracting the correct answer from a "needle in a haystack." The additional chunks also increase embedding and storage costs.
- Recursive chunking underperforms both fixed and semantic strategies. With lower recall@1 (0.467) and MRR (0.590), it presents no advantage over semantic chunking. This strategy should be avoided.

A caveat on generality: the analysis above uses BGE-M3 as the reference model, since it's the recommended embedding model in this report (see Section 4). However, chunking strategy interacts with model architecture — the chart above shows that for bge-large and instructor-xl, recursive chunking actually outperforms both fixed-size and semantic on recall@1, the opposite of the pattern seen with bge-m3. Readers using a different embedding model should consult the full model-by-strategy chart rather than assume BGE-M3's pattern generalizes. If you're following this report's recommendation to use BGE-M3, the "avoid recursive" guidance holds; if using a different model, check that model's column in the chart above before ruling out recursive chunking.

3.2 Precision Considerations for RAG

Precision@k by Model



The precision metrics reveal an important tradeoff:

Strategy	BGE-M3 Precision@1	BGE-M3 Precision@5	BGE-M3 Precision@10
Fixed	0.556	0.164	0.087
Recursive	0.467	0.151	0.087
Semantic	0.556	0.164	0.087

With exactly one relevant chunk per query, $\text{Precision}@k = \text{Recall}@k / k$ by definition. The low precision values are artifacts of the single-relevant-document evaluation design and would be substantially higher in a production RAG system where multiple chunks can be relevant to a query. However, the tradeoff is clear: larger chunks may inflate recall by including the answer somewhere, at the cost of delivering more irrelevant text. Semantic chunking mitigates this by keeping chunks more focused.

3.3 Strategy Recommendation

Semantic chunking is the recommended default strategy. It matches the retrieval ceiling of fixed-size chunking while producing more focused chunks that reduce the LLM's extraction burden. The cost of additional chunks is acceptable for most RAG systems, and semantic chunking better handles diverse document structures.

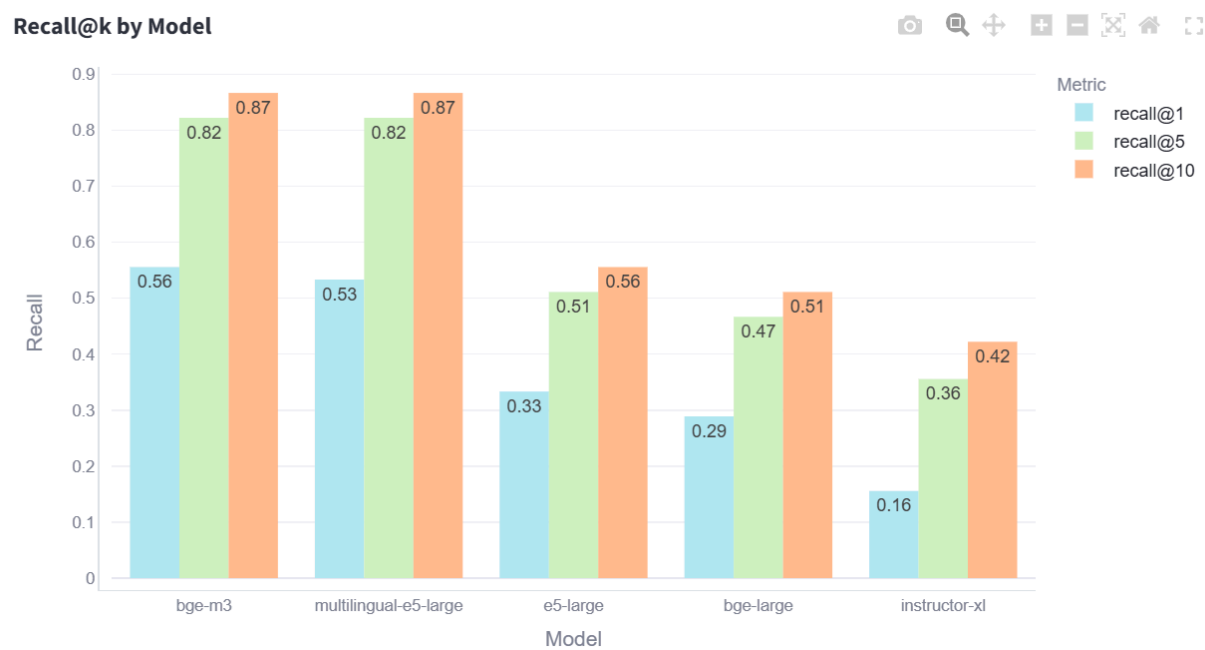
Fixed-size chunking is a viable alternative when minimizing embedding and storage costs is paramount, but the lack of semantic boundaries should be considered.

Recursive chunking should be avoided; it offers no measurable advantage.

4. Results: Embedding Models

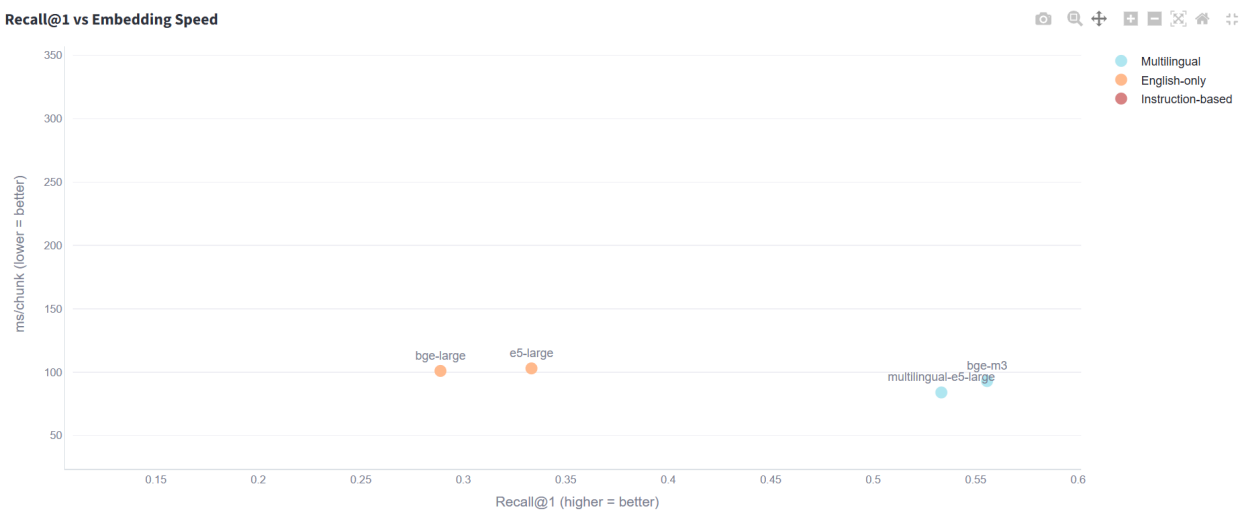
Embedding model selection determines the quality of semantic representation.

4.1 Overall Retrieval Quality



Model	Recall@1	Recall@5	Recall@10	MRR
BGE-M3	0.556	0.822	0.867	0.661
multilingual-e5-large	0.533	0.822	0.867	0.655
E5-large	0.333	0.511	0.556	0.402
BGE-large-en	0.289	0.467	0.511	0.364
Instructor-XL	0.156	0.356	0.422	0.243

Recall@1 vs Embedding Speed

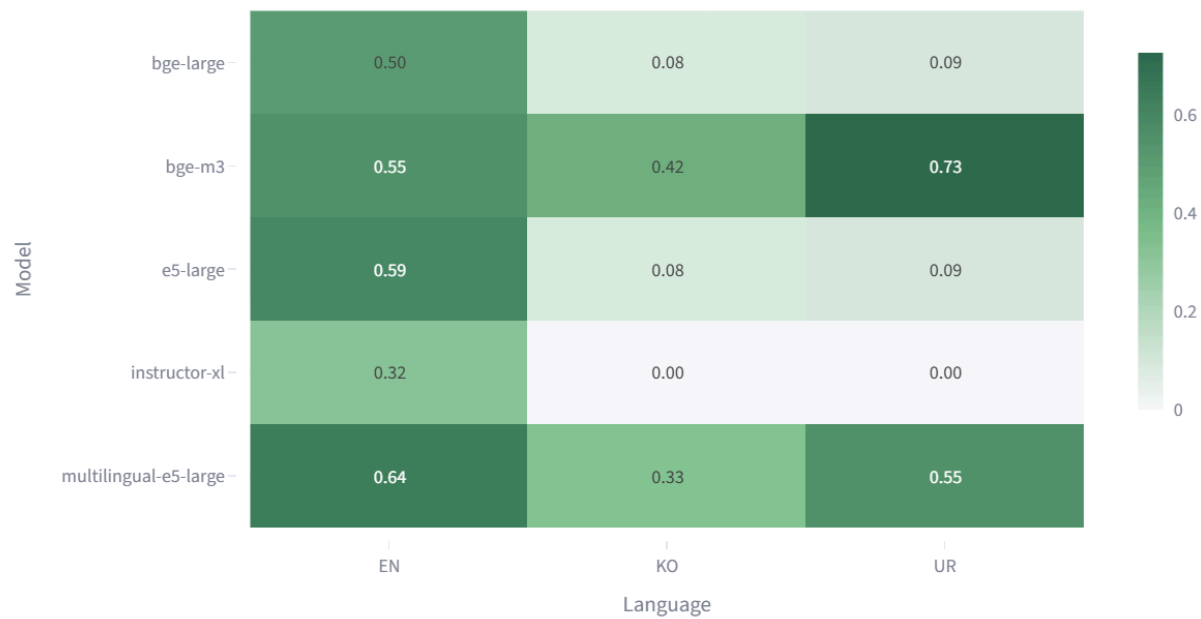


Key observations:

- BGE-M3 is the strongest performer overall. It achieves the highest recall@1 (0.556) and MRR (0.661), meaning it is most likely to place the correct chunk at the top position.
- multilingual-e5-large performs nearly identically to BGE-M3, with a slight edge in English recall@1 (0.636 vs 0.545) but weaker performance on Korean.
- The English-only models (E5-large, BGE-large-en) suffer catastrophic degradation on non-English queries, dropping to 8–9% recall. They are unsuitable for multilingual RAG.
- Instructor-XL is strictly dominated: it has the lowest recall and the highest embedding time (364 ms/chunk, roughly 4× slower than all other models). It should never be used for this task.

4.2 Performance by Language

Recall@1 Heatmap — Model vs Language



Model	English Recall@1	Korean Recall@1	Urdu Recall@1
BGE-M3	0.545	0.417	0.727
multilingual-e5-large	0.636	0.333	0.545
E5-large	0.591	0.083	0.091
BGE-large-en	0.500	0.083	0.091
Instructor-XL	0.318	0.000	0.000

Key observations:

- BGE-M3 is the only model that maintains strong performance across all three languages, with its best result on Urdu (0.727).
- multilingual-e5-large excels on English (0.636) but shows a pronounced weakness on Korean (0.333) under semantic chunking.

- English-only models collapse on Korean and Urdu, achieving ≤ 0.091 recall. Any multilingual RAG system must use a multilingual embedding model.

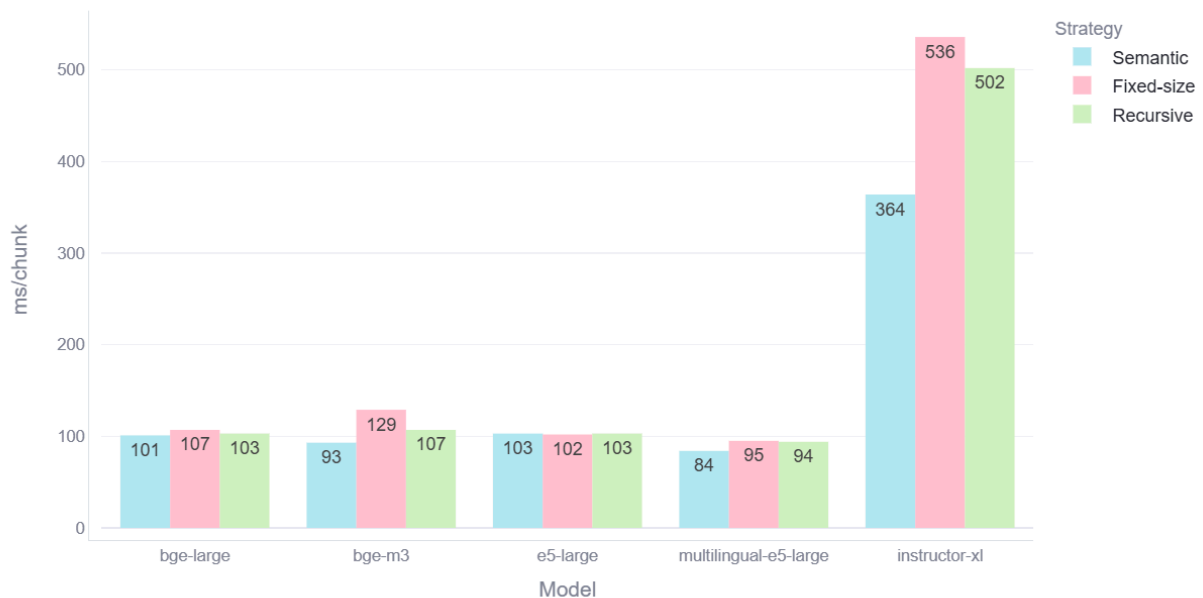
4.3 Precision@k

Precision decays steadily as k increases, which is expected given that each query has only a single relevant chunk. Precision@1 is identical to recall@1 by definition. By $k=10$, precision for the top models drops to approximately 0.09, meaning approximately one in every ten retrieved chunks is the ground-truth chunk.

This is expected behavior for single-relevant-document retrieval. In a production RAG system where multiple chunks can be relevant, precision@ k would be higher.

4.4 Model Recommendation

Embedding Speed (ms/chunk) by Strategy



BGE-M3 is the recommended embedding model for multilingual RAG. It provides the best overall recall, the highest MRR, and consistent performance across all three languages. The slightly faster embedding speed of multilingual-e5-large (84 vs 93 ms/chunk) does not justify the loss in recall on Korean.

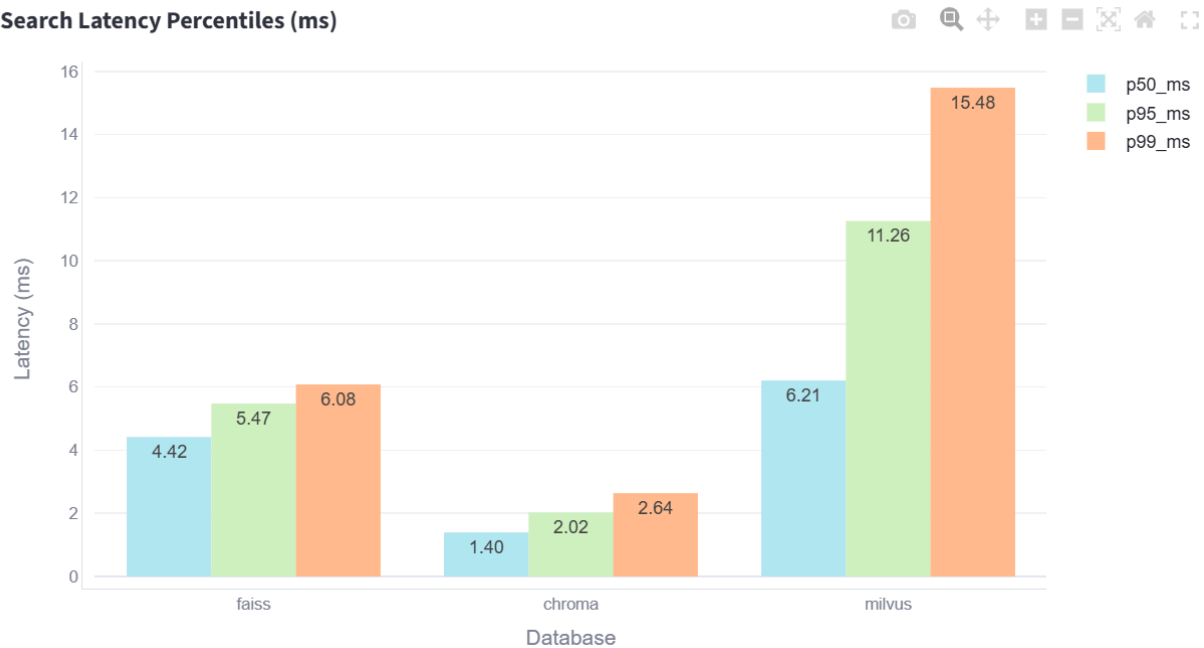
English-only models (E5-large, BGE-large-en) should be avoided for any multilingual use case. Instructor-XL should be avoided entirely due to its poor retrieval quality and extremely slow embedding speed.

5. Results: Vector Databases

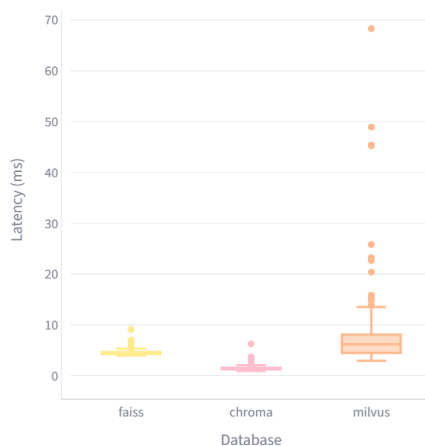
With exact (FLAT) indexes, all three databases return the same top-k results modulo minor tie-breaking differences. Database choice is therefore an operational decision based on latency, build time, memory usage, and infrastructure requirements.

5.1 Search Latency

Search Latency Percentiles (ms)



Latency Distribution



Database	p50 (ms)	p95 (ms)	p99 (ms)
faiss	4.42	5.47	6.08
chroma	1.40	2.02	2.64
milvus	6.21	11.26	15.48

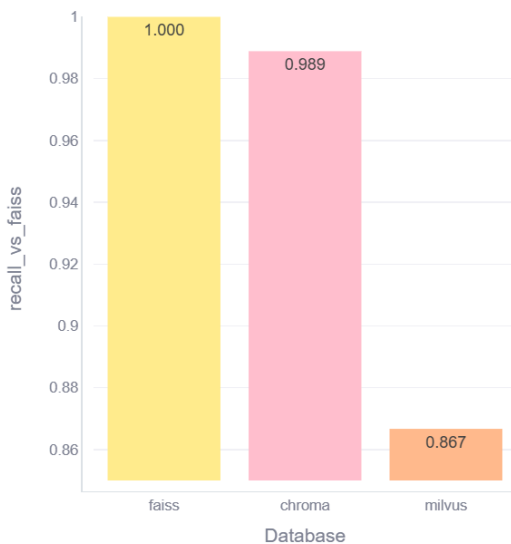
Chroma	1.4	2.5	2.8
FAISS	4.4	5.5	6.1
Milvus	6.2	11.3	15.5

Key observations:

- Chroma is the fastest database by a substantial margin, with p50 latency of 1.4 ms and p99 of 2.8 ms.
- Milvus exhibits the heaviest tail latency, with p99 (15.5 ms) roughly 2.5× its median, reflecting TCP round-trip overhead in Docker containers.
- FAISS provides solid latency (4.4 ms median) with no server overhead.

5.2 Recall vs FAISS Ground Truth

Recall vs FAISS Ground Truth



Database	Recall vs FAISS
Chroma	0.989
FAISS	1.000

Milvus	0.867
--------	-------

Key observations:

- Chroma achieves near-perfect recall (0.989) while also being the fastest database.
- Milvus lags at 0.867 recall vs FAISS. This gap surfaces only when the true 10th- and 11th-best matches are separated by a tiny margin—approximately 0.00002 in cosine similarity, effectively a coin flip. Milvus's segment-based distributed search occasionally picks the "wrong" neighbor in these edge cases, while FAISS's single-pass exhaustive search does not. This is a narrow tie-breaking quirk at the retrieval boundary, not lost or corrupted data.
- With exact FLAT indexes, all three databases produce equivalent results in the vast majority of cases.

5.3 Build Time & Memory

Build time measures the time to insert all 10,456 vectors and construct a searchable index — a one-time cost paid once per database, not per query. Reported values are the median across 5 independent build cycles to reduce noise from transient system load.

Memory is measured as the growth in the benchmarking process's resident memory (RSS) after building each index — accurate for FAISS and Chroma, which run in-process. Milvus runs as a separate Dockerized server, so its actual memory usage is not visible to this measurement; it is not lower than FAISS/Chroma, just unmeasured by this method.

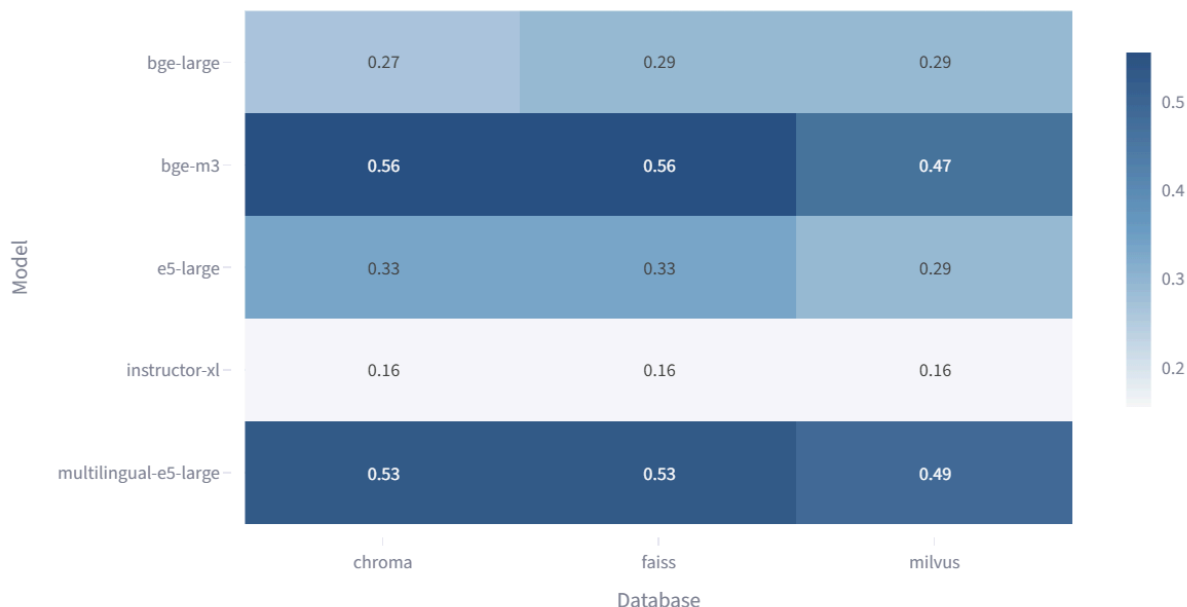
Database	Build Time (s)	Memory (MB)
FAISS	0.011	42.8
Chroma	3.11	58.1
Milvus	7.39	N/A

Key observations:

- FAISS builds the index in milliseconds with the smallest memory footprint.
- Chroma requires 3 seconds to build and uses 58 MB of memory.
- Milvus has the longest build time (7.4 seconds) and memory information is not available (out-of-process).

5.4 Cross-Cut: Model $\times$ Database Interaction

Recall@1 — Model vs Database



Note that this recall@1 gap for Milvus (0.47 vs 0.56 for BGE-M3) is somewhat larger than the top-10 set-overlap metric in Section 5.2 (0.867) might suggest — this is because the two metrics measure different things: Section 5.2's overlap counts how many of the same chunks appear anywhere in the top 10, while recall@1 here specifically checks whether the correct answer landed in the very first position. The tie-breaking behavior described in 5.2 appears to disproportionately affect rank-1 placement for BGE-M3 specifically.

With exact (FLAT) indexes, retrieval quality is determined by the embedding model, not the database. All databases return the same top-k results modulo minor tie-breaking differences. This confirms that embedding model and vector database are independent choices. The best model and the best database can be selected independently without worrying about unexpected interactions.

Retrieval quality is unaffected by database choice here because all three systems were configured to perform the same underlying computation — exhaustive similarity comparison against every stored vector, with no approximation. Differences between databases only emerge when they use approximate indexing structures (like HNSW or IVF) that deliberately trade some accuracy for speed by skipping some vectors during search. At this corpus size, that tradeoff wasn't necessary, so it wasn't tested — which is why this finding should not be read as "vector databases don't affect recall" in general, only as "they don't, when configured for exact search."

5.5 Database Recommendation

Chroma is recommended for production deployments. It offers the fastest query latency, near-perfect recall, and simple embedded deployment with persistence. Chroma is the operational sweet spot.

FAISS is recommended as the baseline for exact search when retrieval quality must be guaranteed without any database approximation or tie-breaking artifacts. It is zero-dependency, fast to build, and memory-efficient.

Milvus offers no advantage at this scale. Its strengths in horizontal scaling and distributed operations only become relevant at much larger scales. For a corpus of this size, Milvus introduces unnecessary latency and complexity.

6. Synthesis and Recommendations

6.1 Hierarchy of Design Decisions

The benchmark results establish a clear hierarchy for retrieval pipeline design:

1. **Chunking Strategy:** Sets the retrieval ceiling. Semantic chunking provides focused chunks with minimal fragmentation. Fixed-size chunking is a cost-saving alternative, but semantic chunking is preferred for RAG to reduce the LLM's extraction burden.
2. **Embedding Model:** Determines semantic representation quality. BGE-M3 is the top performer across languages and provides the highest MRR.
3. **Vector Database:** Operational decision. Chroma provides the best latency, FAISS provides the most reliable exact search baseline.

6.2 Final System Recommendation

For a multilingual RAG system using this corpus, the recommended stack is:

Component	Recommendation	Rationale
Chunking	Semantic	Produces focused, semantically coherent chunks; maximizes retrieval ceiling while maintaining precision for LLM extraction.
Embedding Model	BGE-M3	Highest recall@1, highest MRR, consistent performance across English, Korean, and Urdu.
Vector Database	FAISS	Provides exact search without database-induced recall loss or tie-breaking artifacts. Zero-dependency, fast builds, memory-efficient.

FAISS is preferred over Chroma for the final system because it guarantees retrieval quality is determined solely by the embedding model and chunking strategy, not by database implementation details. FAISS's exact search establishes a reliable baseline that can be validated and reproduced. Chroma, while faster and operationally simpler, introduces minor recall degradation (0.989 vs 1.000) and is recommended only when query latency is the primary operational constraint.

6.3 Production Alternatives

Priority	Recommendation	Rationale
Quality (default)	Semantic + BGE-M3 + FAISS	Maximum retrieval ceiling, guaranteed exact search, no database artifacts.
Latency	Semantic + BGE-M3 + Chroma	~3× faster queries than FAISS, near-perfect recall, simple persistent storage.
Cost	Fixed + BGE-M3 + FAISS	Fewer chunks reduce embedding and storage costs; fixed-size still achieves matching recall@10.

6.4 Components to Avoid

Component	Reason
Recursive chunking	Lower recall and MRR than both fixed and semantic; no compensating advantage.
Instructor-XL	Strictly dominated: worst recall, slowest embedding (4× slower than alternatives).
English-only models	Collapse to <10% recall on Korean and Urdu. Unsuitable for multilingual RAG.
Milvus	No advantage at this scale; slower queries, longer build time, lower recall vs FAISS.

6.5 Corpus-Specific Caveats

The following caveats should be considered when applying these recommendations:

1. Korean queries are near-exact substrings: Korean test queries are held-out sentences from the corpus, not natural questions. This makes the task easier (substring matching) and Korean scores should not be directly compared to English absolute scores.
2. Single relevant chunk per query: Precision@k is artificially low because each query has only one correct answer. In a real RAG system where multiple chunks can be relevant, precision would be higher.
3. Exact indexes only: Approximate indexes (HNSW, IVF) were not tested. At 10k vectors, exact is the right choice. At 1M+ vectors, the tradeoff story would change.
4. GPU embedding speed: Reported embedding timings reflect GPU throughput (Google Colab). CPU-only deployments should expect embedding, not retrieval, to be the dominant time cost.
5. Urdu concentration: Urdu recall scores reflect retrieval from only 3 source documents; this concentration should be considered when interpreting language-level averages.

7. Conclusion

For a 10,456-chunk multilingual corpus spanning English, Korean, and Urdu, the optimal retrieval system uses semantic chunking, the BGE-M3 embedding model, and FAISS as the vector database. This combination provides the highest retrieval quality, with BGE-M3 achieving 0.556 recall@1, 0.867 recall@10, and 0.661 MRR.

FAISS is recommended as the database because it provides exact search without recall degradation or tie-breaking artifacts, ensuring retrieval quality is determined solely by the chunking and embedding components. Chroma is a faster, simpler alternative for latency-sensitive production deployments, with only minor recall degradation (0.989 vs 1.000).

The embedding model choice matters more than the database choice—BGE-M3 and multilingual-e5-large are the top performers, while Instructor-XL is strictly dominated. Chunking strategy is the highest-leverage design decision, as it sets the ceiling on retrieval quality and affects downstream LLM extraction. Semantic chunking is recommended for RAG applications due to its focused, semantically coherent chunks.